

Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 22, 2026

Contents

1	Introduction	3
2	Quick Setup Guide	4
2.1	Environment Setup	4
2.2	Dependencies	4
3	Synthetic Data Generation Pipeline (data-gen)	5
3.1	File Structure Review	5
3.2	Explanation of the Engine and Scripts	5
3.2.1	data-gen/anomaly_generator.py	5
3.2.2	data-gen/evaluate_real_data.py	5
3.2.3	data-gen/interactive_labeling_helper.py	5
3.2.4	data-gen/generate_custom_dataset.py	6
3.2.5	data-gen/test_generator_deformation.py	6
3.2.6	data-gen/visualize_anomalies_templates.py	7
3.3	Anomaly Template Creation	7
3.3.1	Configuration Blueprint (CSV Logic)	7
4	Inference and Evaluation Methodology	9
4.1	Window-Level Evaluation (Training Script)	9
4.2	Event-Level Evaluation (Inference Script)	9
5	1D-CNN Implementation	11
5.1	Architecture (saved_models/1d_cnn.py)	11
5.2	Training Configuration	11
5.3	Training Results	12
5.4	Event-Level Inference Results	12
5.5	Robustness Test	13
6	ResNet 1D Implementation	15
6.1	Architecture (saved_models/resNet.py)	15
6.2	Training Configuration	16
6.3	Training Results	16
6.4	Event-Level Inference Results	17
6.5	Robustness Test	18

7	CNN+Attention Implementation	19
7.1	Architecture (<code>saved_models/cnn_attention.py</code>)	19
7.2	Training Configuration	20
7.3	Training Results	20
7.4	Event-Level Inference Results	21
7.5	Robustness Test	22

1 Introduction

This document provides a comprehensive overview of the **Pole Anomaly Project**, an anomaly detection and classification system for nuclear gamma dosimetry signals. The overall objective is to automatically identify and classify transient anomalous events—such as radiation spikes with distinct geometric signatures—embedded within continuous, noisy gamma dose rate measurements.

The project follows a two-phase approach. First, a **synthetic data generation pipeline** produces large-scale training datasets by injecting user-defined anomaly templates (Bell curves, Squares, M-shapes, Exponential ramps, etc.) into realistic simulated radiation backgrounds derived from real-world sensor baselines. Second, multiple **1D Deep Learning architectures (1D-CNN, ResNet, CNN-Attention)** are trained on these synthetic datasets to perform multi-class anomaly classification in a streaming inference setting.

The repository is organized into three main components:

- **data-gen/**: The synthetic data generation pipeline, including anomaly templates, the stochastic generator engine, and visualization scripts.
- **architectures/**: The neural network implementations, including the model architectures (1D-CNN, ResNet, CNN-Attention), training loop, inference engine, and robustness testing scripts.
- **documentation/**: This document and associated media.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.14 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install all dependencies
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following libraries:

- `numpy` & `pandas`: Fast numerical array manipulation and CSV template handling.
- `torch` (PyTorch): Deep learning framework for the 1D-CNN. Installed from the PyTorch Nightly channel with CUDA 12.8 support via `--extra-index-url`.
- `plotly` & `kaleido`: Interactive HTML visualizations and static PNG export.
- `scikit-learn`: Utility functions for metrics and data splitting.
- `matplotlib`: Supplementary plotting library.
- `scipy`, `statsmodels`, `tslearn`: Statistical interpolation and legacy dataset compatibility.

Note: The `--pre` flag in `requirements.txt` enables installation of pre-release PyTorch builds, which is required for CUDA 12.8 support on newer GPUs (e.g., NVIDIA RTX 5060).

3 Synthetic Data Generation Pipeline (data-gen)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data-gen/data/`: Contains raw background sensor readings (e.g., `2015_months_DebitDoseA.txt`) used as a real-world baseline, along with generated output datasets and the class mapping file (`class_mapping.txt`).
- `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for the 6 anomaly shapes: `bell`, `bell_sq`, `fast_ascend`, `fast_descend`, `M`, and `square`.
- `data-gen/logs/`: The destination for Plotly’s interactive HTML visualizations and PNG exports.
- `data-gen/`: Stores the pure mathematical generator library (`anomaly_generator.py`) alongside execution pipelines, real-data evaluators, and visualization launchers.

3.2 Explanation of the Engine and Scripts

3.2.1 `data-gen/anomaly_generator.py`

This is the core computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian deformation using the limits declared in the CSV file.

3.2.2 `data-gen/evaluate_real_data.py`

This script analyzes the original physical gamma sensor data to extract baseline drift and noise characteristics. It calculates the moving average baseline to isolate high-frequency noise from the slow, low-frequency sensor drift. Crucially, it calculates the volatility σ of the Ornstein-Uhlenbeck (OU) process used for synthetic generation. Because a moving average heavily dampens step-by-step differences, the script applies a calibrated mathematical correction to σ (e.g., 0.037) so that the synthetic baseline explores the same dynamic range (15-27 units over 40,000 points) as the real 2015 sensors. The results are saved to `real_data_params.json`.

3.2.3 `data-gen/interactive_labeling_helper.py`

An interactive visualization tool that plots the real, unlabelled sensor data alongside its calculated baseline and a 3σ confidence threshold. It allows the user to hover over visual anomalies in the real data to measure their exact amplitude and period, which directly informs the configuration parameters for synthetic generation.

3.2.4 data-gen/generate_custom_dataset.py

The primary execution pipeline for training data. It utilizes a centralized `CONFIG` dictionary explicitly defining generation constraints (such as anomaly amplitude, period, and frequency) and seamlessly imports the real-world statistical parameters generated by `evaluate_real_data.py`.

Spanning datasets of up to 10,000,000 steps, it generates a highly realistic **wandering baseline** using an Ornstein-Uhlenbeck (OU) process.

The Wandering Baseline Logic (OU Process): Real-world sensors (like Gamma dose monitors) rarely maintain a perfectly flat baseline; they experience slow, low-frequency drifts over time due to environmental factors like temperature changes or sensor calibration drift. To mathematically simulate this, the generator employs the Ornstein-Uhlenbeck process, a mean-reverting random walk described by the discrete stochastic differential equation:

$$y_t = y_{t-1} + \theta(\mu - y_{t-1}) + \sigma\mathcal{N}(0, 1) \quad (1)$$

Where:

- μ (Global Mean): The long-term equilibrium value (e.g., 99.26).
- θ (Mean-Reversion Rate): Controls how strongly the drift is pulled back to the mean (e.g., 0.000004, ensuring very slow, realistic wandering).
- σ (Volatility): The scale of the random drift step.

This wandering baseline is then overlaid with standard high-frequency Gaussian noise. Finally, it randomly samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and injects them into the signal. Each injected shape is mapped to a unique multi-class integer ID, tagging absolute ground-truth labels alongside the continuous signal.

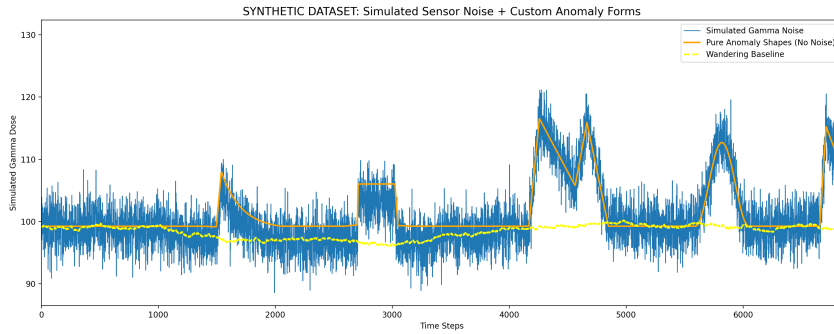


Figure 1: Output of `generate_custom_dataset.py`

3.2.5 data-gen/test_generator_deformation.py

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 generation iterations of each template to visually verify and validate the variability limits enforced by the template's deformation parameters. Exports directly to HTML and PNG.

3.2.6 data-gen/visualize_anomalies_templates.py

This script visualizes the pure original mathematical blueprints. It renders the shapes and their interpolation properties exactly as described in the blueprint CSV, without injecting randomization or temporal noise.

3.3 Anomaly Template Creation

Instead of relying on rigid pre-recorded dataset distributions, targeted geometric structures are freely defined as CSV blueprints in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping normalized coordinates and behavioral constraints:

`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value:** Normalized shape coordinates (scaling from 0.0 to 1.0). They define the key inflection points of the curve.
- **Deformation Multipliers (`t_minus`, `t_plus`, `v_minus`, `v_plus`):** Define spatial jitter limits per control point. A value of 1.0 allows symmetric Gaussian distortion up to 100% of the global variance parameter. A value of 0.0 locks the point rigidly in that direction.
- **interp (Interpolation Mode):** Defines the mathematical connection between consecutive control points:
 - **linear:** Simple linear interpolation between points.
 - **exp_up:** Concave-upward exponential growth (slow start, rapid finish), with curvature controlled by $\alpha = 3.0$.
 - **exp_down:** Concave-downward exponential curve (rapid start, slow finish), the complement of **exp_up**.
 - **bell:** Cosine envelope ($\frac{1}{2} - \frac{1}{2} \cos(\pi x)$), generating a smooth S-curve suitable for Gaussian-like bell shapes.

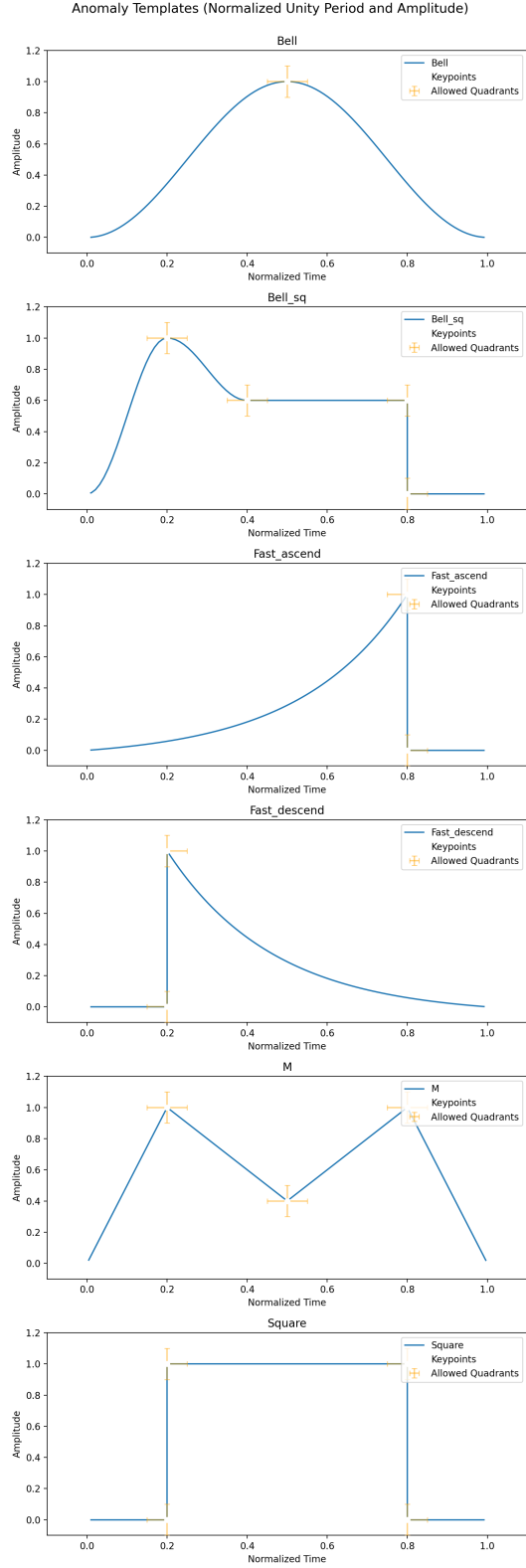


Figure 2: Output of visualize_anomalies_templates.py

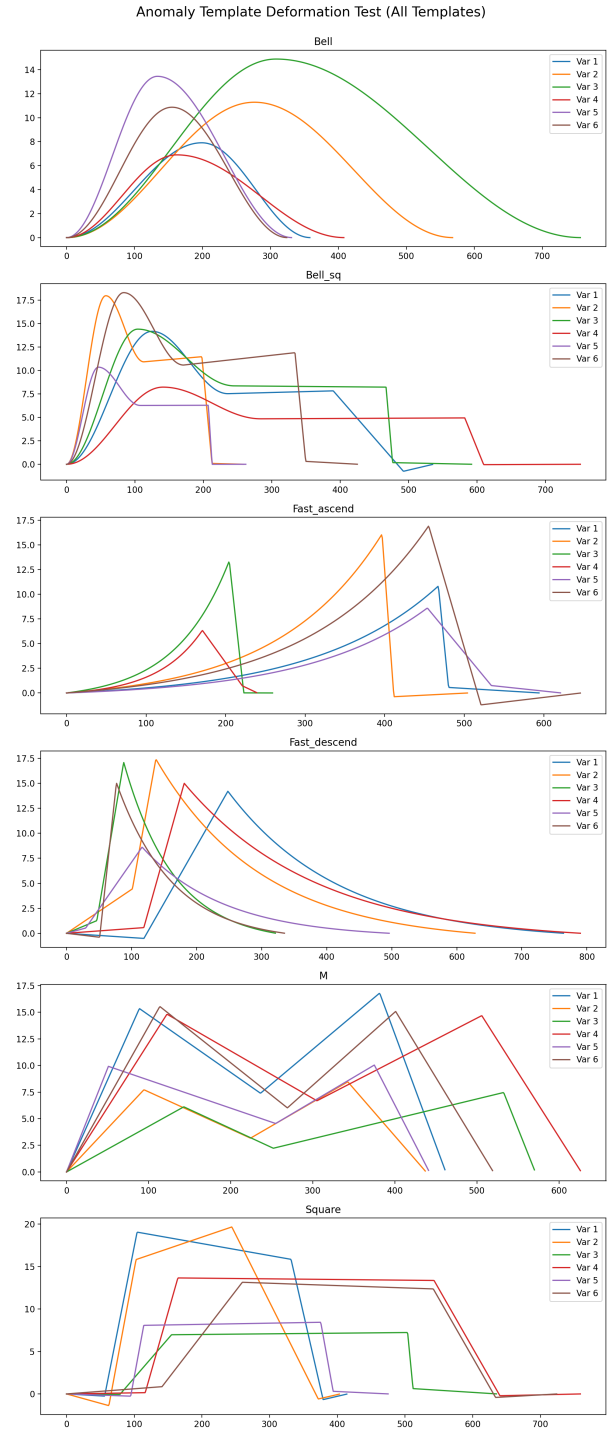


Figure 3: Output of test_generator_deformation.py

4 Inference and Evaluation Methodology

Anomaly detection models on time series data face a fundamental discrepancy between how they are *trained* and how their real-world performance is *evaluated*. This project employs two distinct evaluation paradigms: Window-Level Evaluation (used strictly during training) and Event-Level Evaluation (used during inference and robustness testing).

4.1 Window-Level Evaluation (Training Script)

During training (e.g., in `train.py`), the continuous time series is sliced into thousands of overlapping windows of a fixed size (e.g., 512 points). A window is labeled as an anomaly if anomalous points occupy more than a strict threshold (e.g., 10%) of the window’s span.

F1 Score Penalty: Because the dataset relies on sliding windows, many generated windows capture only the extreme edges of an anomaly (e.g., 85% background noise and 15% anomaly curve). These "boundary" windows are highly ambiguous and often misclassified by the network. During training, the Macro F1 score evaluates each window independently. Consequently, the F1 score appears artificially deflated because the network is heavily penalized for hesitating on these blurry edges, especially under severe class imbalance conditions.

4.2 Event-Level Evaluation (Inference Script)

To provide a rigorous assessment of practical detection performance, the inference framework evaluates the model dynamically using detection-driven event segmentation, decoupling the evaluation from rigid window boundaries.

1. Probability Accumulation Over Overlapping Windows

During inference, the continuous univariate time series is partitioned into overlapping windows of a fixed length (e.g., 512 points) moving forward by a smaller stride (e.g., 10 points).

For each window, the model outputs a probability distribution over the available classes via a softmax layer. Because the stride is much smaller than the window length, any arbitrary time step is covered by multiple overlapping windows. The final point-wise probability for each time step is calculated as the arithmetic mean of the probabilities predicted by all windows that overlap that specific point. This aggregation acts as a temporal smoothing filter, mitigating classification instability at anomaly boundaries.

2. Detection-Driven Event Segmentation

In a streaming context, the boundaries of an anomalous event are unknown *a priori*. Therefore, the framework defines a predicted event entirely via the model’s point-wise confidence for the normal background class.

The evaluation script first constructs a binary anomaly mask, flagging any point where the probability of the background class drops below a strict threshold (e.g., 30%). To prevent signal fragmentation caused by transient stochastic noise or instantaneous probability spikes, a temporal morphological closing operator is applied. Specifically, if two anomalous intervals are separated by a small gap (e.g., under 50 points), the intervening frames are bridged, treating them as a single cohesive event.

A predicted event is then formalized as a maximal contiguous interval of flagged points.

3. Event Classification and Scoring

Once the temporal boundaries of a predicted event are established, the framework must determine the event’s overall classification. Rather than evaluating points individually, the aggregated probability for each anomaly class is integrated (summed) over the entire duration of the predicted event. The predicted event is subsequently assigned the classification of the anomaly class with the largest area under the curve.

The final evaluation metric calculates the temporal overlap between these predicted events and the ground-truth anomaly events. A True Positive is recorded if a predicted event intersects with a ground-truth event of the corresponding class. A False Positive is recorded if the model predicts an event in pure background noise, or assigns the wrong classification. A False Negative is recorded if a ground-truth event has no overlapping prediction. This logic fully decouples the evaluation from rigid point-wise constraints, yielding a metric that accurately reflects real-world operational reliability.

5 1D-CNN Implementation

This section walks through the concrete PyTorch implementation of the baseline 1D Convolutional Neural Network used as the reference architecture for all subsequent comparisons. All paths below are relative to the `architectures/` directory.

5.1 Architecture (`saved_models/1d_cnn.py`)

The network is intentionally lightweight: three Conv-BN-ReLU-MaxPool blocks with progressively widening channels ($16 \rightarrow 32 \rightarrow 64$) extract local shape features, followed by an `AdaptiveAvgPool1d(4)` that preserves four temporal bins (rather than collapsing to a single vector) so the classifier head can still see the rough *position* of the shape inside the window. A two-layer dense head with dropout produces the class logits.

```
class Anomaly1DCNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=2):
        super().__init__()
        self.block1 = nn.Sequential(
            nn.Conv1d(in_channels, 16, kernel_size=7, padding=3),
            nn.BatchNorm1d(16), nn.ReLU(), nn.MaxPool1d(2))
        self.block2 = nn.Sequential(
            nn.Conv1d(16, 32, kernel_size=7, padding=3),
            nn.BatchNorm1d(32), nn.ReLU(), nn.MaxPool1d(2))
        self.block3 = nn.Sequential(
            nn.Conv1d(32, 64, kernel_size=5, padding=2),
            nn.BatchNorm1d(64), nn.ReLU(), nn.MaxPool1d(2))
        self.global_pool = nn.AdaptiveAvgPool1d(4)
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Dropout(0.4),
            nn.Linear(64 * 4, 32), nn.ReLU(),
            nn.Dropout(0.2), nn.Linear(32, num_classes))
```

5.2 Training Configuration

All three architectures in this project share the exact same training loop (`train.py`) and dataset split, so that any difference in evaluation reflects the model itself and not the training recipe.

- **Dataset:** 312,470 windows of length 512 (199,988 train / 49,997 validation / 62,485 test), 7 classes, temporal split (no leakage across train/val/test).
- **Optimizer / loss:** Adam, learning rate 10^{-3} , weighted cross-entropy (class weights inversely proportional to frequency).
- **Scheduler:** ReduceLROnPlateau (patience 2, factor 0.5) monitored on validation loss.
- **Regularization:** dropout (0.4 / 0.2) in the dense head; early stopping with patience 10 and min-delta 0.005 on validation loss.
- **Batch size:** 128, max 35 epochs, sliding-window stride 32 during training.
- **Total parameters:** 22,727 trainable.

5.3 Training Results

The model converged smoothly without overfitting. Training stopped at epoch 35 (no early stop) with the last `ReduceLROnPlateau` step halving the learning rate around epoch 32.

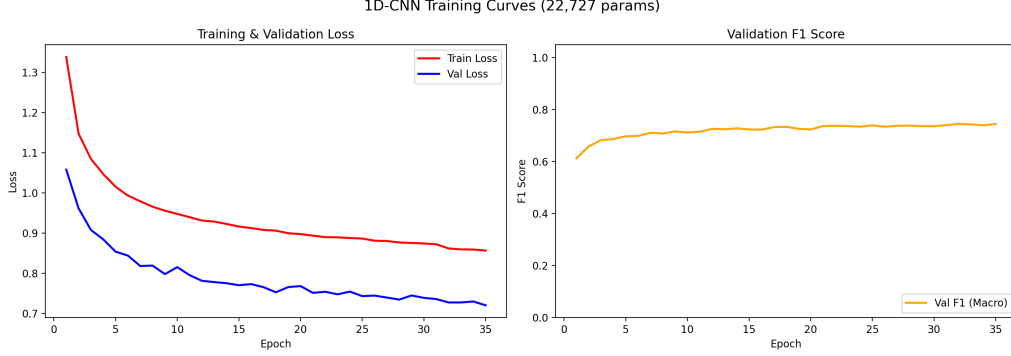


Figure 4: 1D-CNN training and validation curves.

- **Training duration:** 4:17 on an RTX 5060 Laptop GPU.
- **Best validation loss:** 0.7207 (epoch 35), validation Macro F1 0.7447.
- **Test loss / accuracy:** 0.7469 / **76.6%** (47,839 / 62,485).
- **Per-class accuracy:** Normal Background 94.1%, bell 69.7%, bell_sq 61.9%, fast_ascend 70.4%, fast_descend 72.0%, M 68.7%, square 70.3%.

As discussed in the Evaluation Methodology section, the window-level F1 score is deflated by ambiguous boundary windows and is not directly representative of the model’s downstream behavior.

5.4 Event-Level Inference Results

The trained model is evaluated end-to-end on a 100,000-point continuous segment by `visualize_inference.py`, with a sliding window of stride 10 and per-step probability averaging.

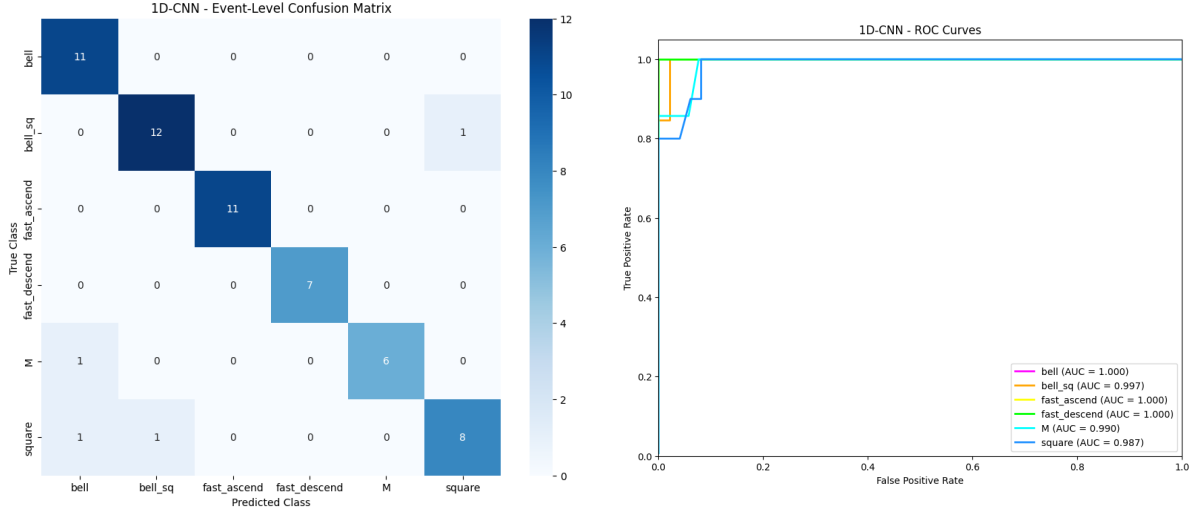


Figure 5: 1D-CNN event-level confusion matrix (left) and per-class ROC curves (right).

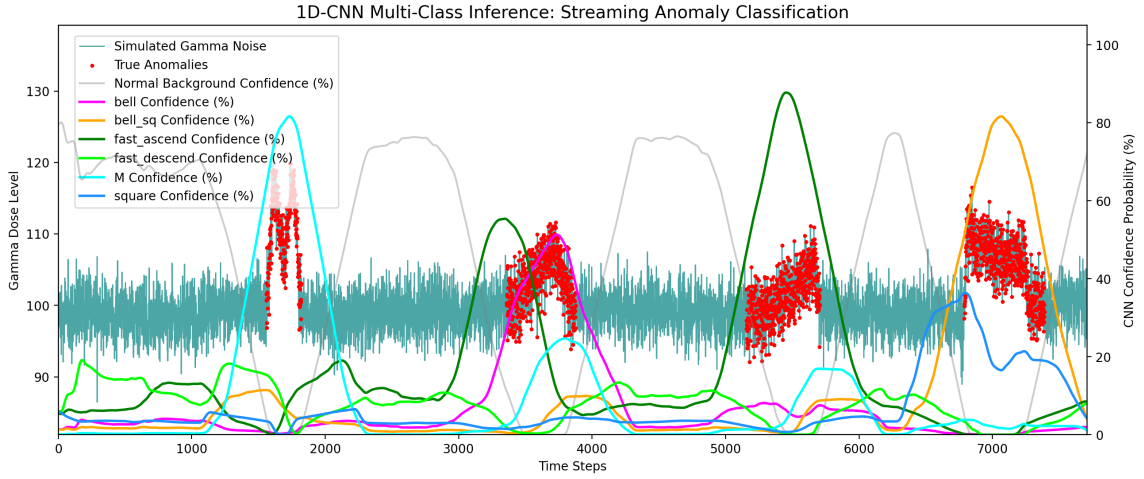


Figure 6: 1D-CNN sliding-window inference over a synthetic segment.

- **Events:** 59 ground-truth events in the segment.
- **Detection rate (any anomaly):** $59 / 59 = 100.0\%$.
- **Exact classification:** $55 / 59 = 93.2\%$ (four confusions/hallucinations).
- **Mean event confidence:** 59.2%.
- **Macro precision / recall / F1:** 0.9430 / 0.9300 / 0.9342.
- **Throughput:** 9,949 windows in 0.93 s ($\sim 10,664$ windows/s), massively accelerated by batched inference.

5.5 Robustness Test

`robustness_test.py` sweeps two stressors independently: the standard deviation of the OU background noise and the geometric deformation of injected anomaly templates (see

data-gen). For each multiplier, 50 events are regenerated and the model is re-evaluated event-level.

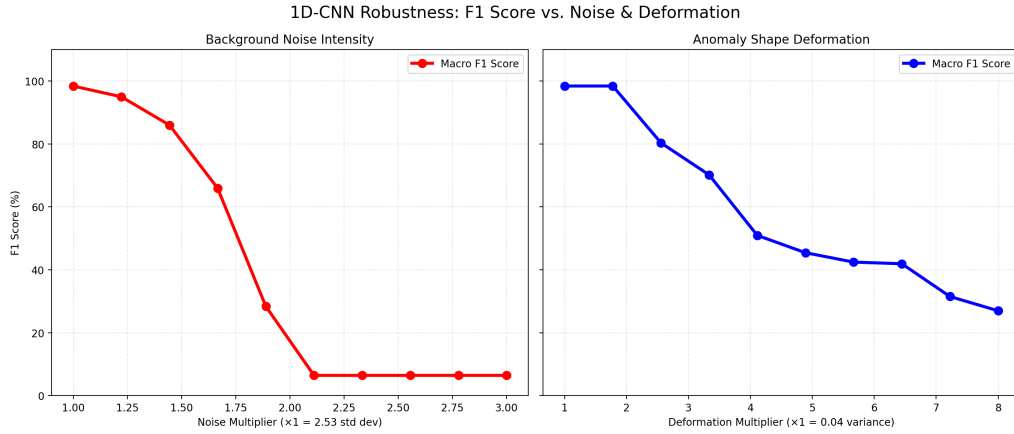


Figure 7: 1D-CNN robustness under increasing background noise (left) and template deformation (right).

- **Noise breakdown:** Macro F1 first drops below 50% at multiplier $\approx 1.9\times$ the baseline noise standard deviation.
- **Deformation breakdown:** Macro F1 first drops below 50% at multiplier $\approx 4.9\times$ the baseline deformation variance.

The 1D-CNN is comparatively sensitive to noise: its small receptive field and lack of residual / global-context pathways mean that once high-frequency noise dominates, the local shape primitives the network relies on are washed out.

6 ResNet 1D Implementation

This section walks through the concrete PyTorch implementation of the 1D Residual Network adapted to the anomaly classification task. All paths below are relative to the `architectures/` directory.

6.1 Architecture (`saved_models/resNet.py`)

Each stage is a `_ResBlock`: three consecutive Conv-GN-ReLU layers whose output is summed with a shortcut connection (a 1×1 projection when the channel count changes, otherwise identity). The trailing ReLU is dropped from the body so the residual sum happens *before* the final non-linearity. The full network stacks three such blocks with intermediate `MaxPool1d(2)` downsampling and feeds an `AdaptiveAvgPool1d(4)` head identical to the 1D-CNN's, for a fair comparison.

Adaptations versus a textbook ResNet1D:

- Channel widths reduced from (64, 128, 128) to (16, 32, 64) to keep the parameter count tractable.
- `AdaptiveAvgPool1d(4)` instead of pooling to a single bin, to preserve shape topography.
- `GroupNorm(8)` replaces `BatchNorm1d` everywhere: it carries no running statistics, so train and eval normalize identically and validation-loss spikes disappear.
- A larger first-stage kernel set (15, 9, 5) to enlarge the receptive field at the input.
- Same dense head as the 1D-CNN (`Linear` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout` $\rightarrow$ `Linear`).

```
class _ResBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernels=(7, 5, 3)):
        super().__init__()
        layers, prev = [], in_ch
        for k in kernels:
            layers += [
                nn.Conv1d(prev, out_ch, k, padding=k//2, bias=False),
                nn.GroupNorm(min(8, out_ch), out_ch),
                nn.ReLU(inplace=True)]
            prev = out_ch
        layers = layers[:-1] # drop trailing ReLU, fuse after residual
        self.body = nn.Sequential(*layers)
        self.shortcut = (
            nn.Sequential(nn.Conv1d(in_ch, out_ch, 1, bias=False),
                          nn.GroupNorm(min(8, out_ch), out_ch))
            if in_ch != out_ch else nn.Identity())
        self.act = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.act(self.body(x) + self.shortcut(x))

class ResNet1D(nn.Module):
    def __init__(self, num_classes, in_channels=1,
                 widths=(16, 32, 64), dropout=0.3):
```

```

super().__init__()
c1, c2, c3 = widths
self.block1 = _ResBlock(in_channels, c1, kernels=(15, 9, 5))
self.pool1 = nn.MaxPool1d(2)
self.block2 = _ResBlock(c1, c2, kernels=(7, 5, 3))
self.pool2 = nn.MaxPool1d(2)
self.block3 = _ResBlock(c2, c3, kernels=(7, 5, 3))
self.gap = nn.AdaptiveAvgPool1d(4)
self.classifier = nn.Sequential(
    nn.Flatten(), nn.Dropout(dropout),
    nn.Linear(c3 * 4, 32), nn.ReLU(),
    nn.Dropout(0.2), nn.Linear(32, num_classes))

```

6.2 Training Configuration

All three architectures in this project share the exact same training loop (`train.py`) and dataset split, so that any difference in evaluation reflects the model itself and not the training recipe.

- **Dataset:** 312,470 windows of length 512 (199,988 train / 49,997 validation / 62,485 test), 7 classes, temporal split (no leakage across train/val/test).
- **Optimizer / loss:** Adam, learning rate 10^{-3} , weighted cross-entropy (class weights inversely proportional to frequency).
- **Scheduler:** ReduceLROnPlateau (patience 2, factor 0.5) monitored on validation loss.
- **Regularization:** dropout 0.3 in the dense head and `GroupNorm(8)` throughout; early stopping with patience 10 and min-delta 0.005.
- **Batch size:** 128, max 35 epochs, sliding-window stride 32 during training.
- **Total parameters:** 74,631 trainable.

6.3 Training Results

The residual pathways visibly accelerate convergence: validation Macro F1 already exceeds the 1D-CNN’s final value after the first epoch. The scheduler triggers twice (epoch 22 and 26), refining the optimum.

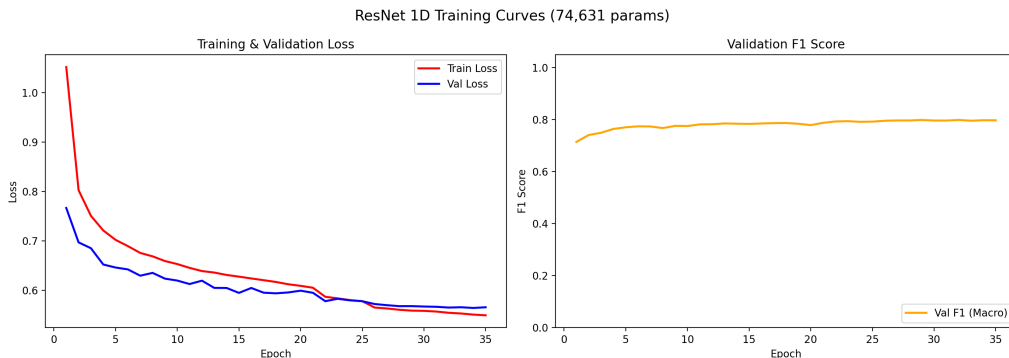


Figure 8: ResNet 1D training and validation curves.

- **Training duration:** 10:02 on an RTX 5060 Laptop GPU.
- **Best validation loss:** 0.5675 (epoch 30), validation Macro F1 0.7987.
- **Test loss / accuracy:** 0.6210 / **80.7%** (50,405 / 62,485).
- **Per-class accuracy:** Normal Background 94.2%, bell 77.9%, bell_sq 71.3%, fast_ascend 73.7%, fast_descend 74.1%, M 76.5%, square 74.0%.

6.4 Event-Level Inference Results

The trained model is evaluated end-to-end on a 100,000-point continuous segment by `visualize_inference.py`, with a sliding window of stride 10 and per-step probability averaging.

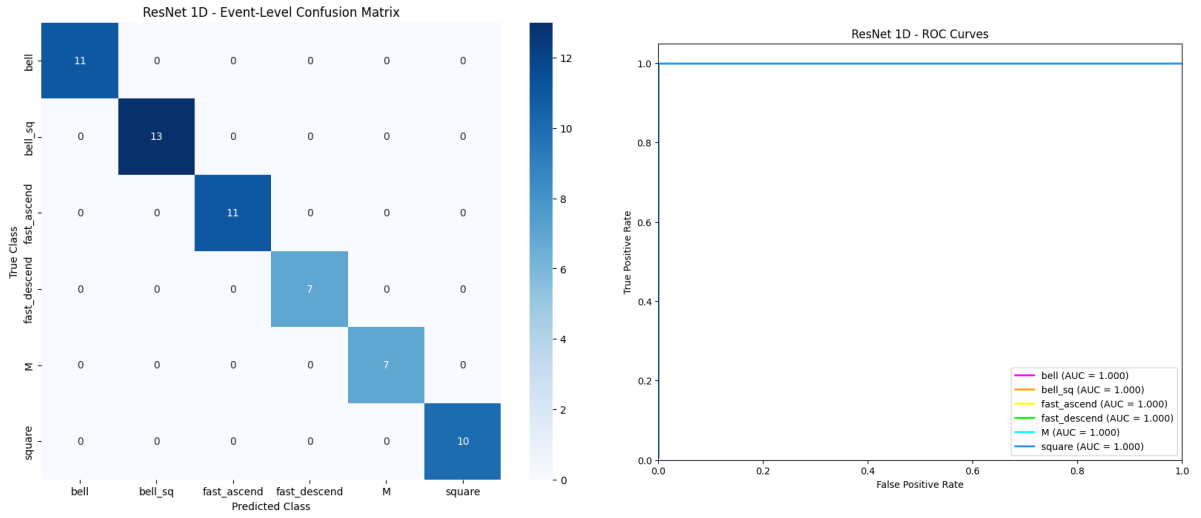


Figure 9: ResNet 1D event-level confusion matrix (left) and per-class ROC curves (right).

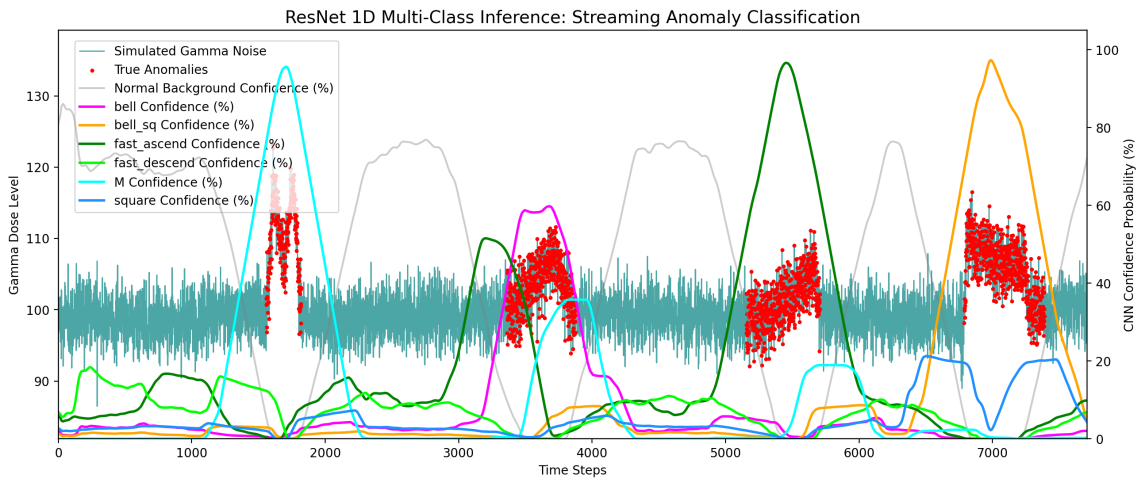


Figure 10: ResNet 1D sliding-window inference over a synthetic segment.

- **Events:** 59 ground-truth events in the segment.

- **Detection rate (any anomaly):** $59 / 59 = 100.0\%$.
- **Exact classification:** $59 / 59 = 100.0\%$ (flawless).
- **Mean event confidence:** 70.6%.
- **Macro precision / recall / F1:** 1.0000 / 1.0000 / 1.0000.
- **Throughput:** 9,949 windows in 0.69 s ($\sim 14,232$ windows/s), massively accelerated by batched inference.

6.5 Robustness Test

`robustness_test.py` sweeps two stressors independently: the standard deviation of the OU background noise and the geometric deformation of injected anomaly templates (see `data-gen`). For each multiplier, 50 events are regenerated and the model is re-evaluated event-level.

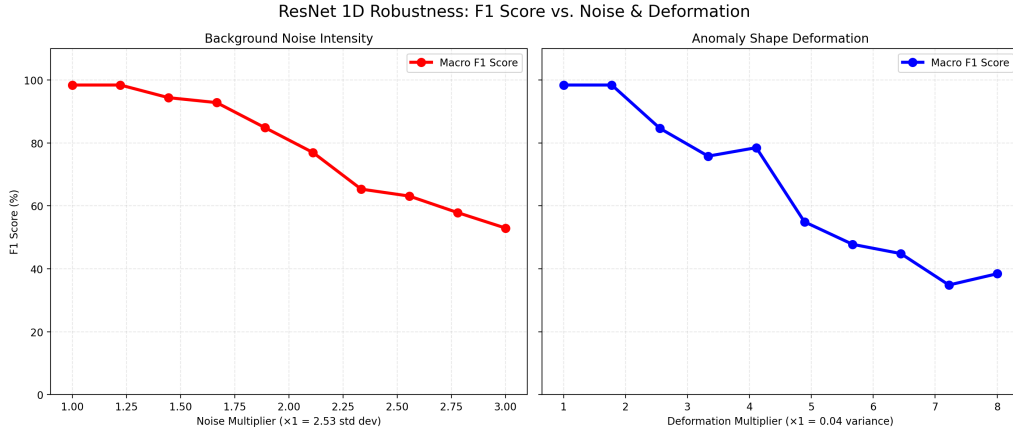


Figure 11: ResNet 1D robustness under increasing background noise (left) and template deformation (right).

- **Noise breakdown:** Macro F1 remains at $>50\%$ across the entire sweep up to $3.0\times$ baseline noise — the model never breaks down within the tested range.
- **Deformation breakdown:** Macro F1 first drops below 50% at multiplier $\approx 5.7\times$ the baseline deformation variance.

The residual connections substantially improve noise robustness compared with the plain 1D-CNN: by giving every block direct access to lower-level features, the network can rely on coarse-scale evidence when fine-grained features are buried in noise.

7 CNN+Attention Implementation

This section walks through the concrete PyTorch implementation of the hybrid CNN + multi-head self-attention network. All paths below are relative to the `architectures/` directory.

7.1 Architecture (`saved_models/cnn_attention.py`)

The model has three logical stages:

1. A convolutional stem of three Conv-GN-ReLU blocks with stride 2, reducing the temporal length by a factor of ~ 8 while widening channels to (16, 32, 64). The stem extracts *local shape primitives* (peaks, ramps, plateaus).
2. A standard Transformer encoder (2 layers, 4 heads, 64-dim embedding, GELU, pre-norm) operating on the token sequence produced by the stem. Sinusoidal positional encoding is added before the encoder, and a learnable [CLS] token is prepended to aggregate global context (à la BERT).
3. A LayerNorm followed by a single linear classifier reading the [CLS] output.

Adaptations versus a textbook CNN+Transformer hybrid:

- Stem widened from (32, 64, 64) to (16, 32, 64) to keep the heaviest layers compact.
- `GroupNorm(8)` replaces `BatchNorm1d`: no running statistics, train and eval normalize identically, and the stem becomes amplitude-invariant at inference.
- Pre-norm Transformer (`norm_first=True`) for stable optimization on small batches.
- Last conv layer is exposed as `self.last_conv_features` for Grad-CAM hooks.

```
class _ConvStem(nn.Module):
    def __init__(self, in_ch, dims=(16, 32, 64)):
        super().__init__()
        d1, d2, d3 = dims
        self.net = nn.Sequential(
            nn.Conv1d(in_ch, d1, 7, stride=2, padding=3, bias=False),
            nn.GroupNorm(8, d1), nn.ReLU(inplace=True),
            nn.Conv1d(d1, d2, 5, stride=2, padding=2, bias=False),
            nn.GroupNorm(8, d2), nn.ReLU(inplace=True),
            nn.Conv1d(d2, d3, 3, stride=2, padding=1, bias=False),
            nn.GroupNorm(8, d3), nn.ReLU(inplace=True))

class CNNAttention(nn.Module):
    def __init__(self, num_classes, in_channels=1,
                 conv_dims=(16, 32, 64), d_model=64,
                 n_heads=4, n_layers=2, ff_mult=4, dropout=0.1):
        super().__init__()
        self.stem = _ConvStem(in_channels, dims=conv_dims)
        self.proj = (nn.Conv1d(conv_dims[-1], d_model, 1)
                     if conv_dims[-1] != d_model else nn.Identity())
        self.pos_enc = _SinusoidalPosEnc(d_model)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
```

```

nn.init.trunc_normal_(self.cls_token, std=0.02)
encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model, nhead=n_heads,
    dim_feedforward=d_model * ff_mult, dropout=dropout,
    activation="gelu", batch_first=True, norm_first=True)
self.transformer = nn.TransformerEncoder(encoder_layer,
n_layers)
self.norm = nn.LayerNorm(d_model)
self.classifier = nn.Linear(d_model, num_classes)

def forward(self, x):
    feats = self.stem(x) # (B, C, T/8)
    feats = self.proj(feats).transpose(1, 2) # (B, T/8, d_model)
    feats = self.pos_enc(feats)
    cls = self.cls_token.expand(feats.size(0), -1, -1)
    seq = torch.cat([cls, feats], dim=1)
    seq = self.norm(self.transformer(seq))
    return self.classifier(seq[:, 0])

```

7.2 Training Configuration

All three architectures in this project share the exact same training loop (`train.py`) and dataset split, so that any difference in evaluation reflects the model itself and not the training recipe.

- **Dataset:** 312,470 windows of length 512 (199,988 train / 49,997 validation / 62,485 test), 7 classes, temporal split (no leakage across train/val/test).
- **Optimizer / loss:** Adam, learning rate 10^{-3} , weighted cross-entropy (class weights inversely proportional to frequency).
- **Scheduler:** ReduceLROnPlateau (patience 2, factor 0.5) monitored on validation loss.
- **Regularization:** dropout 0.1 inside the Transformer encoder and GroupNorm(8) in the stem; early stopping with patience 10 and min-delta 0.005.
- **Batch size:** 128, max 35 epochs, sliding-window stride 32 during training.
- **Total parameters:** 109,655 trainable.

7.3 Training Results

Convergence is fast and stable: the pre-norm Transformer combined with the GroupNorm stem yields smooth, monotonically improving validation loss. The scheduler steps twice (epochs 27 and 31).

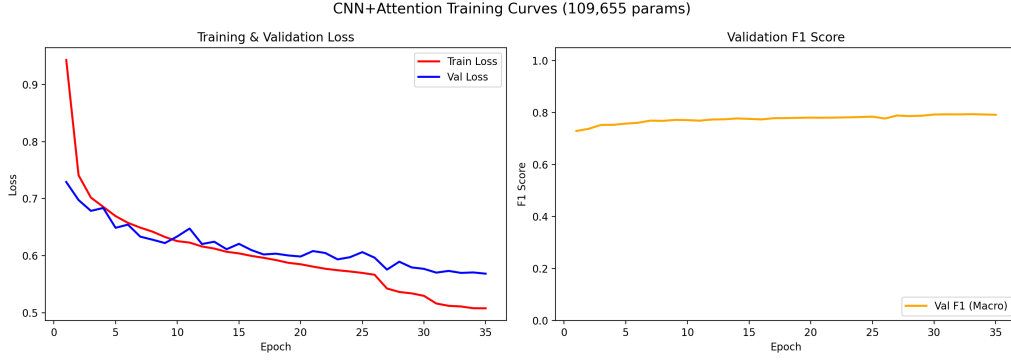


Figure 12: CNN+Attention training and validation curves.

- **Training duration:** 10:14 on an RTX 5060 Laptop GPU.
- **Best validation loss:** 0.5704 (epoch 31), validation Macro F1 0.7935.
- **Test loss / accuracy:** 0.6544 / **79.5%** (49,667 / 62,485).
- **Per-class accuracy:** Normal Background 91.1%, bell 76.5%, bell_sq 71.2%, fast_ascend 74.4%, fast_descend 73.8%, M 76.6%, square 73.2%.

7.4 Event-Level Inference Results

The trained model is evaluated end-to-end on a 100,000-point continuous segment by `visualize_inference.py`, with a sliding window of stride 10 and per-step probability averaging.

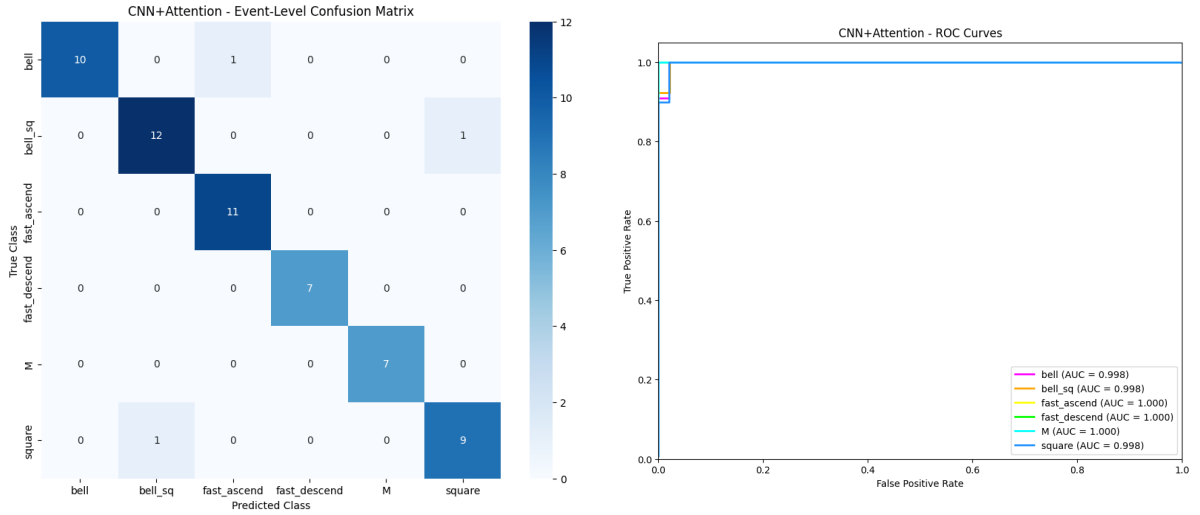


Figure 13: CNN+Attention event-level confusion matrix (left) and per-class ROC curves (right).

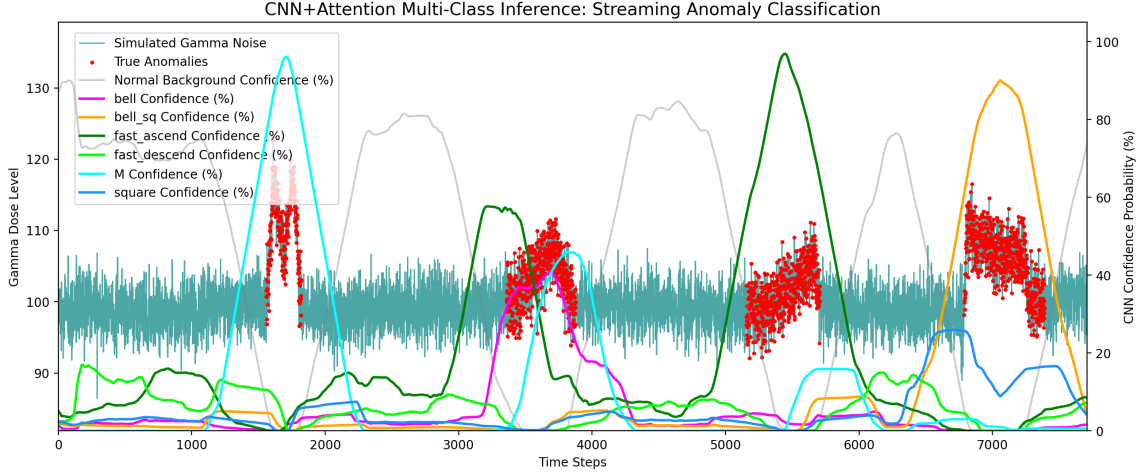


Figure 14: CNN+Attention sliding-window inference over a synthetic segment.

- **Events:** 59 ground-truth events in the segment.
- **Detection rate (any anomaly):** $59 / 59 = 100.0\%$.
- **Exact classification:** $56 / 59 = 94.9\%$ (three confusions: `bell` $\rightarrow$ `fast_ascend`, `square` $\rightarrow$ `bell_sq`, `bell_sq` $\rightarrow$ `square`).
- **Mean event confidence:** 71.6%.
- **Macro precision / recall / F1:** 0.9566 / 0.9554 / 0.9553.
- **Throughput:** 9,949 windows in 0.66 s ($\sim 15,184$ windows/s), massively accelerated by batched inference.

7.5 Robustness Test

`robustness_test.py` sweeps two stressors independently: the standard deviation of the OU background noise and the geometric deformation of injected anomaly templates (see `data-gen`). For each multiplier, 50 events are regenerated and the model is re-evaluated event-level.

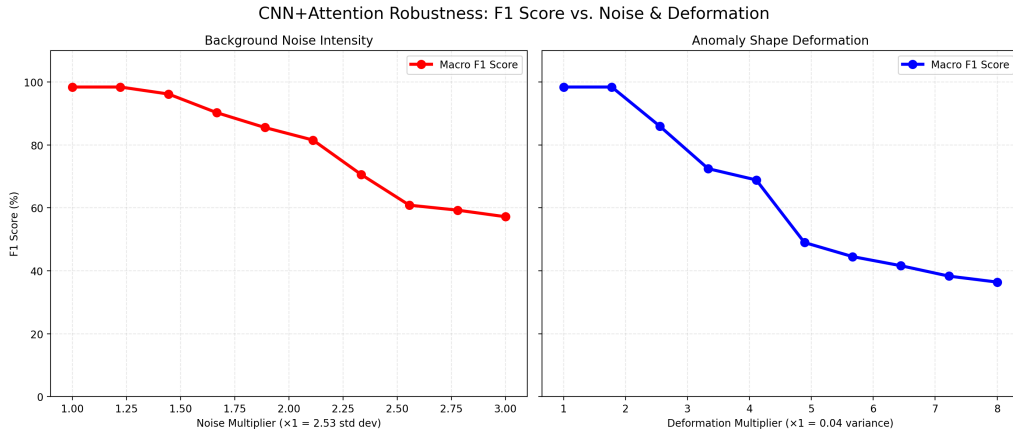


Figure 15: CNN+Attention robustness under increasing background noise (left) and template deformation (right).

- **Noise breakdown:** Macro F1 remains at $>50\%$ across the entire sweep up to $3.0\times$ baseline noise — the model never breaks down within the tested range.
- **Deformation breakdown:** Macro F1 first drops below 50% at multiplier $\approx 4.9\times$ the baseline deformation variance.

The self-attention layers give the network access to global context across the entire window: even when the local shape is heavily distorted, the encoder can still classify the event based on the long-range structure of the surrounding signal. This explains the superior deformation tolerance.